```
])->notify(new orderRecieved($order));
```

Mail Notifications

If a notice can be sent as an email, the notification class should include a **`toMail`** function. This function should return an **`IlluminateNotificationsMessagesMailMessage`** instance after receiving a **`$notifiable`** entity.

The **`MailMessage`** class has a few simple methods for creating transactional email messages. Lines of text and a call to action can be included in email communications. Let's look at an example of the **`toMail`** method:

```
public function toMail(object $notifiable): MailMessage
{
 $url = url('/order/'.$this->invoice->id);
 return (new MailMessage)
     ->greeting('Hello!')
     ->line('We have recieved your order')
     ->lineIf($this->amount > 0, "Amount paid: {$this-
     >amount}")
     ->action('View Bill', $url)
     ->line('Thank you for using our application!');
}
```

In this example, we have a welcome, a line of text, a call to action, and another line of text. The **`MailMessage`** object's methods make it simple and quick to prepare tiny transactional emails. The message components will subsequently be translated by the mail channel into a beautiful, responsive HTML email design with a plain-text equivalent.

Instead of setting the *lines* of text in the notification class, you may use the view function to define a custom template for rendering the notification email:

```
public function toMail(object $notifiable): MailMessage
{
 return (new MailMessage)->view(
  'emails.name', ['order' => $this->order]
 );
}
```